Netflix Movies & TV Shows

Data Analysis Web Application

Technical Documentation

Category	Details
Project Type	End-to-End SLM Web Application
AI Model	Google Gemini 2.5 Flash
Database	SQLite (9,837 movie records)
Frontend	React.js
Backend	FastAPI (Python)
Deployment	Vercel (Frontend) + Render (Backend)
MCP Integration	Gemini Function Calling (get_schema, query_database)

1. Project Overview

This project is an end-to-end Small Language Model (SLM) web application that allows users to query a movie database using natural language. The system converts user questions into SQL queries using Google Gemini AI, executes them against a SQLite database, and returns human-readable answers through a chat-style interface.

The application is built with a modern full-stack architecture consisting of a React frontend, a FastAPI Python backend, and a SQLite database pre-loaded with 9,837 Netflix movie and TV show records sourced from Kaggle.

---

2. Architecture

2.1 System Flow

The following describes how a user query is processed end-to-end:

- User types a natural language question in the React UI
- Frontend sends a POST request to the FastAPI backend at /ask
- Backend passes the question to Gemini AI with MCP-style function tools
- Gemini calls get_schema to understand the database structure
- Gemini generates SQL and calls query_database to execute it
- SQLite runs the query and returns raw results
- Gemini formats the results into a readable answer
- Backend returns the answer to the frontend
- React UI displays the response in the chat window

2.2 Tech Stack

Layer	Technology	Purpose
Frontend	React.js	Chat UI, user input, response display
Backend	FastAPI (Python)	API server, Gemini integration, DB queries
Database	SQLite	Stores 9,837 movie records
AI Model	Google Gemini 2.5 Flash	Natural language to SQL conversion
MCP Layer	Gemini Function Calling	Tool interface between AI and database
Frontend Host	Vercel	Free static hosting with auto-deploy
Backend Host	Render	Free Python web service hosting
Version Control	GitHub	Source code and deployment trigger

---

3. MCP Integration

Model Context Protocol (MCP) is implemented via Gemini's function calling feature. Instead of a formal MCP server, the application defines two tools that Gemini can invoke during a conversation:

3.1 get_schema

This tool returns the database schema including table names and column definitions. Gemini always calls this first before generating any SQL query to understand the data structure.

```
Function: get_schema()
Returns: Table structure with column names and types
Called by: Gemini AI automatically before writing SQL
```

3.2 query_database

This tool accepts a SQL query string, executes it against the SQLite database, and returns the results as a list of rows with column headers.

```
Function: query_database(sql: str)
Returns: { columns: [...], rows: [...] }
Called by: Gemini AI after schema inspection
```

3.3 Agentic Loop

The backend implements an agentic loop where Gemini can call multiple tools in sequence before producing a final answer. The loop runs up to 5 iterations and stops when Gemini produces a text response with no further tool calls.

---

4. Dataset

4.1 Source

The dataset was sourced from Kaggle and contains Netflix movie and TV show data. It was downloaded as a CSV file and loaded into a SQLite database using pandas.

4.2 Schema

Column	Type	Description
release_date	TEXT	Date the movie/show was released
title	TEXT	Title of the movie or TV show
overview	TEXT	Plot summary or description
popularity	REAL	Popularity score
vote_count	INTEGER	Number of user votes
vote_average	REAL	Average rating out of 10
original_language	TEXT	Language code (en, fr, es, etc.)

genre	TEXT	Comma-separated genres
poster_url	TEXT	URL to the movie poster image

4.3 Statistics

- Total records: 9,837
- Columns: 9
- Format: SQLite database (movies.db)
- Source: Kaggle - mymoviedb.csv

---

5. Project Structure

Path	Description
movie-query-app/	Root project folder
backend/	Python FastAPI server
backend/main.py	Main API server with Gemini integration
backend/load_data.py	Script to load CSV into SQLite
backend/requirements.txt	Python dependencies
backend/.env	Environment variables (API key)
frontend/	React application
frontend/src/App.js	Main React component (UI)
data/	Database files
data/mymoviedb.csv	Original dataset
data/movies.db	SQLite database
.gitignore	Files excluded from GitHub

---

6. API Reference

POST /ask

The single API endpoint that accepts a natural language question and returns an AI-generated answer.

Request

```
URL: https://movie-query-app-gjzy.onrender.com/ask
Method: POST
Content-Type: application/json
```

```
Body: { "question": "your question here" }
```

Response

```
{ "answer": "Gemini formatted response here" }
```

Example

```
Question: "What are the top 5 highest rated movies?"
Answer: "Here are the top 5 highest rated movies: 1. ..."
```

---

7. Deployment

7.1 Backend - Render

- Platform: render.com (free tier)
- Runtime: Python 3
- Start command: uvicorn main:app --host 0.0.0.0 --port 10000
- Environment variable: GEMINI_API_KEY
- Auto-deploys on every GitHub push
- Note: Free tier sleeps after inactivity (50s cold start)

7.2 Frontend - Vercel

- Platform: vercel.com (free tier)
- Framework: React (Create React App)
- Root directory: frontend/
- Auto-deploys on every GitHub push
- Live URL: https://movie-query-app.vercel.app

7.3 Running Locally

Backend

```
cd backend
venv\Scripts\activate
uvicorn main:app --reload
```

Frontend

```
cd frontend
npm start
```

---

8. Features

8.1 Core Features

- Natural language to SQL query conversion using Gemini AI

- MCP-style function calling with get_schema and query_database tools
- Real-time chat interface with hot pink and black theme
- Persistent chat history stored in browser localStorage
- Timestamps on every message
- Clear history button
- Suggested sample questions on first load
- Animated typing indicator while waiting for response

8.2 Sample Queries

Question Type	Example Query
Top N	Give me the top 5 highest rated movies
Filter by genre	List all horror movies with rating above 7
Aggregation	What is the average rating by genre?
Language filter	Which movies are not in English?
Date filter	Show movies released after 2020
Popularity	What are the most popular Action movies?
Count	How many movies are in each genre?

---

9. Dependencies

9.1 Backend (Python)

Package	Version	Purpose
fastapi	latest	Web framework for the API server
uvicorn	latest	ASGI server to run FastAPI
google-genai	latest	Google Gemini AI SDK
pandas	latest	CSV loading and data manipulation
python-dotenv	latest	Load API key from .env file
python-multipart	latest	Form data support for FastAPI
sqlite3	built-in	Database engine (Python standard library)

9.2 Frontend (Node.js)

Package	Purpose
react	UI component library
react-dom	DOM rendering

react-scripts	Build tooling (Create React App)

10. Known Limitations

- Free tier Render backend sleeps after inactivity, causing 30-50 second cold start delays
- Gemini free tier has rate limits (requests per minute and per day)
- Chat history is stored in browser localStorage only — not synced across devices
- Database is static — no real-time data updates
- Maximum 50 rows returned per query to prevent timeouts

End of Documentation